



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

数据安全实验报告

Lab 1: 半同态加密应用实践

陈语童

年级：2023 级

学号：2311887

专业：计算机科学与技术

课程教师：刘哲理

2026 年 4 月 27 日

目录

一、 实验要求	2
二、 实验原理	2
(一) 同态加密	2
(二) 半同态加密	2
(三) Paillier 算法	3
(四) 对称加密	4
(五) AES 算法	4
三、 环境配置	5
四、 实验过程	5
(一) 基于 Python 的 phe 库完成加标量乘法的验证	5
(二) 隐私信息获取	9
(三) 扩展实验: AES 对称加密	11
五、 实验总结	13

一、实验要求

基础实验：基于 Paillier 算法实现隐私信息获取：从服务器给定的 m 个消息中获取其中一个，不得向服务器泄露获取了哪一个消息，同时客户端能完成获取消息的解密。

扩展实验：有能力的同学，可以在客户端保存对称密钥 k ，在服务器端存储 m 个用对称密钥 k 加密的密文，通过隐私信息获取方法得到指定密文后能解密得到对应的明文。

二、实验原理

（一）同态加密

同态加密 (Homomorphic Encryption) 是一种允许 **在密文上直接进行运算** 的加密技术，解密后的结果等价于对明文进行相同运算的结果。根据支持运算类型不同，可分为半同态加密和全同态加密。

其关键在于“能在密文上直接运算”。传统的加密流程是：

明文 $\rightarrow$ 加密 $\rightarrow$ 密文 $\rightarrow$ 解密 $\rightarrow$ 明文 $\rightarrow$ 再计算

而同态加密的一般流程是：

明文 $\rightarrow$ 加密 $\rightarrow$ 密文上直接计算 $\rightarrow$ 解密 $\rightarrow$ 得到正确结果

背后的本质是一种保持运算结构的映射（同态）。

一般同态加密的分类有：

1. 部分同态加密。只支持一种同态运算：

- 加法同态：加密函数 E 满足 $E(m_1) \cdot E(m_2) = E(m_1 + m_2)$ 。即对两个明文 m_1 和 m_2 分别加密后得到的密文进行乘法运算，其结果进行解密后，与对 m_1 和 m_2 先进行加法运算的结果一致。
- 乘法同态：加密函数 E 满足 $E(m_1)^k = E(k \cdot m_1)$ 。即对明文 m_1 的密文进行 k 次幂运算，其结果进行解密后，与将 m_1 乘以 k 的结果一致。

2. 有限同态加密：只能支持有限种、或有限次数运算的同态加密方案。即可以做加法和乘法，但计算复杂度或运算深度受到限制（噪声增长）。

3. 全同态加密：能够支持任意多次数的加法和乘法。

（二）半同态加密

半同态加密 (Semi-Homomorphic Encryption) 是同态加密的一种 **特殊/受限** 版本。“半同态”意味着只对一种运算保持同态性，它的特点是：**只支持一种类型的同态** 运算（加法或乘法），但可以 **无限次进行** 该运算。

半同态加密主要基于特定的 **数学困难问题** 来构造加密算法，使得在密文上能够进行某种运算，并且运算结果解密后与在明文上进行相应运算的结果一致。

以加法同态为例，如果要将两个密文进行加法同态加密，基本流程是：

- 用加密函数 E 分别对明文 m_1 和 m_2 进行加密，得到密文 $c_1 = E(m_1)$ 和 $c_2 = E(m_2)$ 。

- 在密文上进行特定的运算, 前面提到过加法同态应满足 $E(m_1) \cdot E(m_2) = E(m_1 + m_2)$, 故将密文进行乘法得到 $c = c_1 \cdot c_2$.
- 用解密函数 D 对密文 c 解密, 根据加法同态的性质, $D(c) = D(c_1 \cdot c_2) = D(E(m_1 + m_2)) = m_1 + m_2$, 即: 解密后的结果, 与在明文上进行加法运算再加密、再解密后的最终结果是一致的。

半同态加密的优势在于: 相比全同态加密, 其计算效率更高适合工程落地, 且实现较简单, 有清晰的数学结构, 不需要复杂的“噪声管理”(来源于其稳定性)。

半同态加密的局限在于: 只能做一种同态运算, 有一些计算需求无法满足, 比如无法直接计算多项式或逻辑电路。

(三) Paillier 算法

Paillier 算法是一种加法同态加密算法, 基于数论算法, 适合做安全求和、隐私统计、PIR 等工作。下面阐述其具体实现步骤。

前置说明:

- $\mathbb{Z}_{n^2}^*$: 模 n^2 的乘法群
- $\mathbb{Z}_n$: 加法群

具体过程:

1. 密钥生成:

- 选取两个大素数 p 和 q .
- 计算两者乘积 $n = p \cdot q$, 以及 Carmichael 函数 $\lambda = \text{lcm}(p-1, q-1)$.
- 选择生成元 g , 满足 $g \in \mathbb{Z}_{n^2}^*$, 且 $\text{gcd}(L(g^\lambda \bmod n^2), n) = 1$, 其中函数 $L(u) = \frac{u-1}{n}$. 一个常用的特例是 $g = n + 1$.
- 计算 $\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$.
- 密钥构成:
 - 公钥: (n, g)
 - 私钥: (λ, μ)

2. 加密算法:

- 对于明文 $m \in \mathbb{Z}_n$, 选取随机数 $r \in \mathbb{Z}_{n^2}^*$.
- 密文定义为: $c = g^m \cdot r^n \bmod n^2$.

3. 解密算法:

- 计算 $u = c^\lambda \bmod n^2$.
- 应用之前的 $L(u) = \frac{u-1}{n}$.
- 对于密文 c , 计算得到解密结果 $m = L(u) \cdot \mu \bmod n$.

4. 同态性验证:

- 两个明文 m_1 和 m_2 , 其对应的密文分别为 $c_1 = g^{m_1} r_1^n \bmod n^2$ 和 $c_2 = g^{m_2} r_2^n \bmod n^2$.
- 计算 $c = c_1 c_2 \bmod n^2 = (g^{m_1} r_1^n)(g^{m_2} r_2^n) \bmod n^2 = g^{m_1+m_2} (r_1 r_2)^n \bmod n^2$.
- 对 c 进行解密, 如果得到的结果为 $m_1 + m_2$, 就能够验证 Paillier 算法的加法同态性。

(四) 对称加密

对称加密 (Symmetric Encryption) 是最基础、也最常用的一类加密方式。它的核心特点是：**加密和解密使用同一个密钥。**

算法的流程如下：

加密：明文 + 密钥 → 加密 → 密文
解密：密文 + 同一密钥 → 解密 → 明文

其常见的分类有：

1. 分组密码：把明文按固定长度分组，每一组独立或按模式处理。
2. 流密码：不分组，逐比特或逐字节加密。
3. 迭代型：对消息多轮重复变换。
4. 非迭代型（较少见）。

其中 **分组密码** 必须配合不同的“模式 (Mode of Operation)”：

1. ECB (电子密码本模式)：每个分组独立加密。
2. CBC (链式模式)：每个分组依赖前一个密文。
3. CTR (计数器模式)：把分组加密变成“流密码”。
4. GCM (认证加密模式)：加密 + 完整性验证 (MAC)，HTTPS 常用。

(五) AES 算法

AES (高级加密标准) 是目前世界上最重要、使用最广泛的对称分组加密算法。它的设计目标是：安全、高效、可实现硬件加速，并且易于标准化部署。

下面简单介绍 AES 的处理机制。

首先，AES 不是一次变换，而是 **多轮“混合操作”**：

明文 → 初始轮 AddRoundKey → Nr-1 轮标准轮函数 → 最终轮 (略有不同) → 密文

轮数取决于密钥长度：

密钥长度	轮数
128 bit	10 轮
192 bit	12 轮
256 bit	14 轮

AES 每一轮包含四个核心步骤：

1. **SubBytes (字节替换)**：非线性变换，每个字节通过 S-box 查表替换，能够打破线性关系、防止代数攻击。
2. **ShiftRows (行移位)**：纯粹的位置变换：
 - 第 0 行：不动

- 第 1 行：左移 1 字节
- 第 2 行：左移 2 字节
- 第 3 行：左移 3 字节

用来扩散和打散结构。

3. **MixColumns (列混合)**：把每一列当作一个多项式，在有限域 $GF(2)$ 上做矩阵乘法，强扩散作用，一个字节影响到整列。
4. **AddRoundKey (轮密钥异或)**：引入密钥做异或操作，控制整个加密过程。

此外，AES 的密钥通过扩展得到——不直接使用原始密钥，而是生成多个轮密钥：

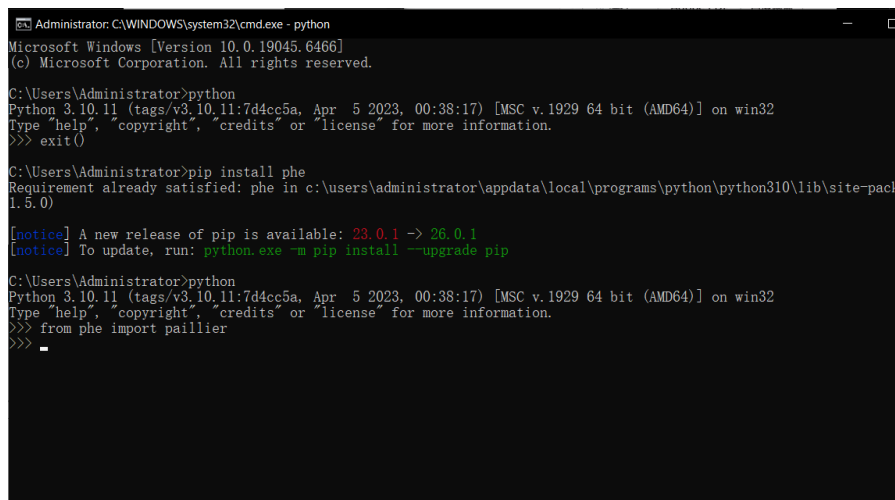
$K_0 \rightarrow K_1 \rightarrow K_2 \rightarrow \dots \rightarrow K_n$

其过程包含：字节替换、循环移位、常量异或，能够防止结构重复并增强抗攻击能力。

AES 在工程上有速度极快、安全性经过长期验证的优点，被广泛使用。但也存在局限性，如需要正确模式（如 CBC / GCM）且不能直接加密流数据（需 CTR 等模式）。

三、 环境配置

实验进行前已安装 python。确认 phe 已安装，验证 phe 包：



```
Administrator: C:\WINDOWS\system32\cmd.exe - python
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator>python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> exit()

C:\Users\Administrator>pip install phe
Requirement already satisfied: phe in c:\users\administrator\appdata\local\programs\python\python310\lib\site-packages (1.5.0)

[notice] A new release of pip is available: 23.0.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\Administrator>python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> from phe import paillier
>>>
```

图 1: 验证环境合规

四、 实验过程

(一) 基于 Python 的 phe 库完成加标量乘法的验证

在 Python IDLE 中打开实验自提供的测试文件 test1.py：

```

test1.py - C:\Users\Administrator\Desktop\STUDY\JUNIOR B\DATA SECURITY\La...
File Edit Format Run Options Window Help

from phe import paillier # 开源库
import time # 做性能测试

##### 设置参数
print("默认私钥大小:", paillier.DEFAULT_KEYSIZE)
#生成公私钥
public_key, private_key = paillier.generate_paillier_keypair()
# 测试需要加密的数据
message_list = [3.1415926, 100, -4.6e-12]

##### 加密操作
time_start_enc = time.time()
encrypted_message_list = [public_key.encrypt(m) for m in message_list]
time_end_enc = time.time()
print("加密耗时s:", time_end_enc-time_start_enc)
print("加密数据 (3.1415926):", encrypted_message_list[0].ciphertext())

##### 解密操作
time_start_dec = time.time()
decrypted_message_list = [private_key.decrypt(c) for c in encrypted_message_list]
time_end_dec = time.time()
print("解密耗时s:", time_end_dec-time_start_dec)
print("原始数据 (3.1415926):", decrypted_message_list[0])

##### 测试加法和乘法同态
a, b, c = encrypted_message_list # a, b, c分别为对应密文
a_sum = a + 5 # 密文加明文, 已经重载了+运算符
a_sub = a - 3 # 密文加明文的相反数, 已经重载了-运算符
b_mul = b * 6 # 密文乘明文, 数乘
c_div = c / -10.0 # 密文乘明文的倒数

print("a+5 密文:", a.ciphertext()) # 密文纯文本形式
print("a+5=", private_key.decrypt(a_sum))
print("a-3=", private_key.decrypt(a_sub))
print("b*6=", private_key.decrypt(b_mul))
print("c/-10.0=", private_key.decrypt(c_div))

##密文加密文
print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a+b))
#报错, 不支持a+b, 即两个密文直接相乘
#print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a*b))

Ln: 1 Col: 0

```

图 2: test1.py

通过 Run -> Run Module, 运行上面的代码:

```

test1.py - C:\Users\Administrator\Desktop\STUDY\JUNIOR B\DATA SECURITY\La...
File Edit Format Run C IDLE Shell 3.10.11
from phe import paillier
import time # 做性能测试

##### 设置参数
print("默认私钥大小:", paillier.DEFAULT_KEYSIZE)
#生成公私钥
public_key, private_key = paillier.generate_paillier_keypair()
# 测试需要加密的数据
message_list = [3.1415926, 100, -4.6e-12]

##### 加密操作
time_start_enc = time.time()
encrypted_message_list = [public_key.encrypt(m) for m in message_list]
time_end_enc = time.time()
print("加密耗时s:", time_end_enc-time_start_enc)
print("加密数据 (3.1415926):", encrypted_message_list[0].ciphertext())

##### 解密操作
time_start_dec = time.time()
decrypted_message_list = [private_key.decrypt(c) for c in encrypted_message_list]
time_end_dec = time.time()
print("解密耗时s:", time_end_dec-time_start_dec)
print("原始数据 (3.1415926):", decrypted_message_list[0])

##### 测试加法和乘法同态
a, b, c = encrypted_message_list # a, b, c分别为对应密文
a_sum = a + 5 # 密文加明文, 已经重载了+运算符
a_sub = a - 3 # 密文加明文的相反数, 已经重载了-运算符
b_mul = b * 6 # 密文乘明文, 数乘
c_div = c / -10.0 # 密文乘明文的倒数

print("a+5 密文:", a.ciphertext()) # 密文纯文本形式
print("a+5=", private_key.decrypt(a_sum))
print("a-3=", private_key.decrypt(a_sub))
print("b*6=", private_key.decrypt(b_mul))
print("c/-10.0=", private_key.decrypt(c_div))

##密文加密文
print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a+b))
#报错, 不支持a+b, 即两个密文直接相乘
#print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a*b))

Ln: 16 Col: 0

```

图 3: Run Module 结果

完整的输出如下：

```
1 默认私钥大小： 3072
2 加密耗时 s： 0.7212095260620117
3 加密数据 (3.1415926) : 227249945253180053103680617681293920355573492453860779
    ↪ 2718067727021024905734938238805917067068958618806746632099063710448404412
    ↪ 9262297215016715369976342987582675495592647363328091188377617917790037288
    ↪ 7615205455237112948957854633256135223464734505453663430417759172439151839
    ↪ 8400187431855643288311992309370792340560455908570908650629228939932349791
    ↪ 1572732996727358397058135466278627206224410127814828777283308165771272051
    ↪ 8512949072980108100700414481681277689376014774609942807961254210137824752
    ↪ 4334354925319490037448813833641434333491225796528508971585133913600988824
    ↪ 2672089431544869894453243147398796501442402194155515496664528499740837759
    ↪ 659738437951750862422253456117743265518443328780821669792957634900764017
    ↪ 8543062947018349784062232783274179227527180687799508968152296634840677237
    ↪ 7870894229307852276600506208567593459123659423666850821126843076521292392
    ↪ 9245581985987513763377628926224239288983647784930226281991035275189871849
    ↪ 3130280250303004180104736358363625265747499440453782184585278712808310520
    ↪ 1188010525330649124712047186008866083492387364731837410500235283849884243
    ↪ 4966442591128442621407369906887383836386062678921416315970885102416767814
    ↪ 9375870381417162330965099679140734257330022983639847551466569854225602699
    ↪ 8937171182767350377770968786677706675884339844487812134571246047763146010
    ↪ 5848377454321038583735831228495089473358692920758344178560380358912871721
    ↪ 8352906804346039341845183366437981958099875044681769053460704283264419185
    ↪ 2913410861749957955520954567901889101110215274265136825836629791481708462
    ↪ 3385954667087235990368626675599376294768483826486897624476042077504549679
    ↪ 3239536744921940234890476631743262657246170764089796921429428414700410259
    ↪ 2795222550942862890132817770811484889983015591243346920763315259161448217
    ↪ 9636356159026509605968728099958548904630661375064308478522690037735882954
    ↪ 33603949195489464520390324527841355197228153
4 解密耗时 s： 0.214616060256958
5 原始数据 (3.1415926) : 3.1415926
```



```

6  a+5 密文: 2272499452531800531036806176812939203555734924538607792718067727021
    ↪ 0249057349382388059170670689586188067466320990637104484044129262297215016
    ↪ 7153699763429875826754955926473633280911883776179177900372887615205455237
    ↪ 1129489578546332561352234647345054536634304177591724391518398400187431855
    ↪ 6432883119923093707923405604559085709086506292289399323497911572732996727
    ↪ 3583970581354662786272062244101278148287772833081657712720518512949072980
    ↪ 1081007004144816812776893760147746099428079612542101378247524334354925319
    ↪ 490037448813833641434334912257965285089715851339136009888242672089431544
    ↪ 8698944532431473987965014424021941555154966645284997408377596597384379517
    ↪ 508624222534561177432655184433287808216697929576349007640178543062947018
    ↪ 3497840622327832741792275271806877995089681522966348406772377870894229307
    ↪ 8522766005062085675934591236594236668508211268430765212923929245581985987
    ↪ 5137633776289262242392889836477849302262819910352751898718493130280250303
    ↪ 0041801047363583636252657474994404537821845852787128083105201188010525330
    ↪ 6491247120471860088660834923873647318374105002352838498842434966442591128
    ↪ 4426214073699068873838363860626789214163159708851024167678149375870381417
    ↪ 1623309650996791407342573300229836398475514665698542256026998937171182767
    ↪ 3503777709687866777066758843398444878121345712460477631460105848377454321
    ↪ 0385837358312284950894733586929207583441785603803589128717218352906804346
    ↪ 0393418451833664379819580998750446817690534607042832644191852913410861749
    ↪ 9579555209545679018891011102152742651368258366297914817084623385954667087
    ↪ 2359903686266755993762947684838264868976244760420775045496793239536744921
    ↪ 9402348904766317432626572461707640897969214294284147004102592795222550942
    ↪ 8628901328177708114848899830155912433469207633152591614482179636356159026
    ↪ 5096059687280999585489046306613750643084785226900377358829543360394919548
    ↪ 9464520390324527841355197228153
7  a+5= 8.1415926
8  a-3 0.14159260000000007
9  b*6= 600
10 c/-10.0= 4.6e-13
11 True

```

对比可知，原始数据和加密后再解密得到的数据是一致的，证明了 Paillier 算法的数理逻辑正确性；对密文做标量的加减乘除后还能解密出正确的信息，证明 Paillier 算法支持线性运算（与明文的线性关系）；两个密文相加还能正确解密，证明 Paillier 算法支持加法同态。

现在将代码最后一行解除注释：

```

1  # 报错，不支持 a*b，即两个密文直接相乘
2  print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a*
    ↪ b))

```

再次 Run -> Run Module，发现会产生报错：

```

Traceback (most recent call last):
  File "C:\Users\Administrator\Desktop\STUDY\JUNIOR B\DATA SECURITY\Lab1\test1.py", line 41, in <module>
    print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a
    *b))
  File "C:\Users\Administrator\AppData\Local\Programs\Python\Python310\lib\site-packages\phe\paillier.py", line 508, in __mul__
    raise NotImplementedError('Good luck with that...')
NotImplementedError: Good luck with that...
>>>
Ln: 35 Col: 0

```

图 4: Run Module 后程序结尾报错

这说明在两个密文上做乘法，Paillier 算法不能够正确解密。这说明，Paillier 算法不支持乘法同态。

故以上实验成功验证了 Paillier 算法只支持加法同态，不支持乘法同态。

(二) 隐私信息获取

此实验要求基于 Python 的 phe 库完成隐私信息获取的功能：服务器端拥有多个数值，要求客户端能基于 Paillier 实现从服务器读取一个指定的数值并正确解密，但服务器不知道所读取的是哪一个。

实验代码如下：

```

test2.py - C:\Users\Administrator\Desktop\STUDY\JUNIOR B\DATA SECURITY\La...
File Edit Format Run Options Window Help

from phe import paillier # 开源库
import random # 选择随机数

##### 设置参数
# 服务器端保存的数值
message_list = [100,200,300,400,500,600,700,800,900,1000]
length = len(message_list)
# 客户端生成公私钥
public_key, private_key = paillier.generate_paillier_keypair()
# 客户端随机选择一个要读的位置
pos = random.randint(0,length-1)
print("要读起的数值位置为:",pos)

##### 客户端生成密文选择向量
select_list=[]
enc_list=[]
for i in range(length):
    select_list.append( i == pos )
    enc_list.append( public_key.encrypt(select_list[i]) )

# for element in select_list:
#     print(element)
# for element in enc_list:
#     print(private_key.decrypt(element))

##### 服务器端进行运算
c=0
for i in range(length):
    c = c + message_list[i] * enc_list[i]
print("产生密文:",c.ciphertext())

##### 客户端进行解密
m=private_key.decrypt(c)
print("得到数值:",m)

Ln: 31 Col: 0

```

图 5: test.py

对这个程序逐步分析：

首先，客户端生成 Paillier 公钥/私钥，并（这里是随机）选择一个目标：

```

1 public_key, private_key = paillier.generate_paillier_keypair()
2 pos = random.randint(0,length-1)

```

而后，客户端根据所选择目标，构造一个“选择向量”，这个向量只有在选择的目標那一位是 1，其余都是 0：

```
1 select_list.append(i == pos)
2 enc_list.append(public_key.encrypt(select_list[i]))
```

比如，选择了 [3]，原始的 select_list 就是：

```
1 select_list = [0, 0, 0, 1, 0, 0, ...]
```

加密后：

```
1 enc_list = [E(0), E(0), E(0), E(1), E(0), ...]
```

关键在于，Paillier 算法是概率加密，虽然数组里有这么多 0，但每个 0 加密结果都不一样，所以对于服务器来说，只能看到一系列互不相同的密文，它并不能分辨哪个是 1，哪些是 0。所以这做到了，对于客户端来说，它选择读取的目标是秘密的，同时服务端不知道客户端读取的数据是哪一个。

接下来还要保证读取数据的正确。服务端用加密后的“选择向量”与数据进行运算：

```
1 c = c + message_list[i] * enc_list[i]
```

展开看就是：

```
1 c = 100 * E(0)
2   + 200 * E(0)
3   + 300 * E(0)
4   + 400 * E(1)
5   + ...
```

最后客户端再对传回的密文 c 进行解密获得要读取的目标值：

```
1 m=private_key.decrypt(c)
```

获取数据的正确性依赖于 Paillier 算法的同态性：

$$E(m_1) \cdot E(m_2) = E(m_1 + m_2)$$

$$E(m_1)^k = E(k \cdot m_1)$$

在上面的实例中，服务器在计算： $c = \sum m_i \cdot E(s_i)$

根据同态性，解密后会变成： $m = \sum m_i \cdot s_i$

由于选择向量 s_i 只有在客户端选择读取的那一位是 1，其余都是 0，所以等价于： $m = m_{target}$ ——解密得到的实际上就是要求读取的那个数据。

整个方案的正确性来源于 Paillier 算法本身的数理逻辑正确，和其加法同态性；对服务器端的不可见性来源于选择项里的设计和 Paillier 算法的概率加密性。运行以上程序验证：



```

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
= RESTART: C:\Users\Administrator\Desktop\STUDY\JUNIOR B\DATA SECURITY\Lab1\test
2.py
要读取的数值位置为: 5
产生密文: 171788249432054707385124452782748499826645307572805012442093428774789
31271862769616742657567600739900108802917743488370368313341380360104820305362622
67074946607962112311060483345717267941250302012389472446201217199934596631814848
15494903636587555796226503971958455428317671989656071279592417937026383264588634
29723033942169777844263845138867092197453425786219503927525136826952978724033976
21960464849530718898619746783726792386990904755731575855692352036025119237615362
15999254393968803752761203951166659817960893375956459562340865272834421819270014
03546465513925340350440443257247474636415174482637773261117563707317330120093998
27915952936937397357250769341635106704124345991815068819490834181993179279900524
53544734584538265009244089098621327950637205506064481628648317000898674833079821
68640732959167526985631186792867560397126507907539549328702333901222090935322398
25400650363927714494674722875952083748688709783558679757835882591851760846316690
78880231405847785270808259474503209427974017908943789138805678138679715177860055
36036364317937479353406558298921985206312682053409439471892258148003090575504805
96840196654390111474786903316362914691492483238368068697623112340482010567694348
95833788152455789380972542767647156462113596052647448327385932325150767606084651
98098313522152706914776582872555659950988587469898956683331688475158411316981866
99009573541043832239893861013389694545489178625323046083412523788424362064039692
07959125508909021005526359895606520929219829161280666198799184983681670398913142
03182653508803395342890709641291152562819620471745792600578389779179274255115628
12016409670049386195064223169907594404944751207019415608878944499181226286856103
12200685955912242004435482342900991360194018315454806914702138117451688254640667
83450428083934753370971032351940112768313204783843706824698220605058523475526049
922206723101352437649
得到数值: 600
>>>
Ln: 40 Col: 0

```

图 6: Run Module 结果

选择的是 [5]，看原始代码中的数组：

```
1 message_list = [100,200,300,400,500,600,700,800,900,1000]
```

第 [5] 位（下标，从 0 开始）是 600，与解密结果一致，说明上面的方案设计是正确的。

(三) 扩展实验：AES 对称加密

扩展实验要求：在客户端保存对称密钥 k，在服务器端存储用对称密钥 k 加密后的密文，通过隐私信息获取方法得到客户端指定密文后能解密得到对应的明文。

查询相关资料后选择用 AES 认证加密算法来实现对称加密。要使用 AES 相关功能函数，首先要安装 pycryptodome 包：

```
pip install pycryptodome
```

导入包：

```
1 from Crypto.Cipher import AES
```

使用 AES 以后，客户端保存对称加密密钥 k，服务端保存的是 AES 算法加密后（同样使用的是密钥 k）的信息列表：

```

1 plain_message_list = [100,200,300,400,500,600,700,800,900,1000]
2 # 客户端保存 AES 密钥 k；服务器只保存下面生成的 AES 密文列表
3 k = b'lab1-symmetric-k'
4 message_list = [symmetric_encrypt(k, i, m) for i, m in
  ↳ enumerate(plain_message_list)]

```

同样地，服务端运算得到的结果，光通过之前 Paillier 算法的解密是不够的，只能得到 AES 密文。客户端得到这个 AES 密文之后还需要用密钥 k 进行 AES 解密：

```
1 ##### 客户端进行解密
2 selected_cipher=private_key.decrypt(c)
3 print("selected ciphertext:",selected_cipher)
4 # Paillier 只负责隐私取回 AES 密文，真正的明文还要再经过 AES 解密
5 m=symmetric_decrypt(k, pos, selected_cipher)
6 print(" 得到数值: ",m)
```

下面是 AES 算法的具体实现。采用 AES-128（体现在 k 的位数），CTR 模式。CTR 模式可以把 AES 变成类似“密钥流”的加密方式，同一个 k 和同一个 nonce（= 配合密钥一起使用的一个公开的一次性参数）会生成同一段密钥流。而在加密时，AES 会把位置 pos 转成 8 字节的 nonce。这样不同位置的数据会使用不同的 AES-CTR 密钥流。以上逻辑写成代码就是：

```
1 # 固定明文字节数
2 BLOCK_SIZE = 16
3
4 def _new_aes_cipher(k, pos):
5     # 用查询位置 pos 生成 nonce，使每个位置使用不同的 AES-CTR 密钥流
6     nonce = pos.to_bytes(8, byteorder='big')
7     return AES.new(k, AES.MODE_CTR, nonce=nonce)
8
9 def symmetric_encrypt(k, pos, message):
10    # 将整数明文固定为 16 字节，再用 AES 加密成密文整数，便于服务器计算
11    message_bytes = int(message).to_bytes(BLOCK_SIZE, byteorder='big')
12    cipher_bytes = _new_aes_cipher(k, pos).encrypt(message_bytes)
13    return int.from_bytes(cipher_bytes, byteorder='big')
14
15 def symmetric_decrypt(k, pos, cipher):
16    # 客户端取回指定位置的密文整数后，用同一个密钥 k 和 pos 解密得到明文
17    cipher_bytes = int(cipher).to_bytes(BLOCK_SIZE, byteorder='big')
18    message_bytes = _new_aes_cipher(k, pos).decrypt(cipher_bytes)
19    return int.from_bytes(message_bytes, byteorder='big')
```

后续代码中直接调用 AES 的加密和解密函数接口即可。除了以上展示部分，其他代码不用改动，保持之前 Paillier 算法保证服务器对客户端选择不可见的机制。

运行程序进行功能验证：

```

test3.py - C:\Users\Administrator\Desktop\STUDY\JUNIOR B\DATA SECURITY\Lab1\test
File Edit Format Run Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Administrator\Desktop\STUDY\JUNIOR B\DATA SECURITY\Lab1\test
3.py
要读起的数值位置为： 8
产生密文： 202695610791412627956465890912899145683990681745152322052227938519496
26254878864107873815211569328385312930438186158360520844887081148616540948191506
56018389214900389491107792819986131262732517136978609706328830070703813240137285
52669008709754930605694021850086079749493749097502287393539862431144841007293816
76471179854833884522929796195891585766849047272317935170525131029895085319885131
79657566139194415390831722059520494771603885295203380458654445904422953707750563
47901509257804076207868077173815771547272201898942129981949875061673063526009057
89930220808834051363581014865115305860148062501750907465537660997206370189766600
30072573719334827301042648195745760791387752283966410412267872381238554036926191
66626803036922735852590045723123719640376418312491192550334502640082940634030810
00852936176273256225389905386947499520479829654793749659777337104831473896175815
5914469296638692660277073506006785824863943440813111682924976450785757582524160
04763511407800149989486367568572376083554690725295191819329685452910411193875387
86273030576091664503774698543140765677839951889755656022801607527846648915216498
42503879329640677025830331471335444716821466814721019090286737788616848861602262
7640826267538162505734900572488178350444300886954500896134363400505953027486534
67181488169680274814418788408762714870741475248799398266712647779965677052588286
30409030183453841166737737877656095959648732057423799637932053856891742457120699
64597262007811339670603801485026235905202570943452821963158218425079302914504894
06141123256165839953242223577843967670092251418940590325870558472736885847762502
61489468055891348467497136456759830769503359350678273356256948861657136900649511
60839896770666422464503380092298368346371085761597017239837059493958997208429215
53756672846856393256696017383774486281187941259005748561093967578596314577659072
30687419899520319335
selected ciphertext: 138873187163970977311164798672680206074
得到数值： 900
>>>
#####
select_list=[]
enc_list=[]
for i in range(length)

```

图 7: Run Module 结果

随机到的位置是 [8]，的导数值是 900，对应原始消息列表的第 [8] 项（从 0 开始计），验证 AES 加密算法实现正确。

五、实验总结

本次实验对于 Paillier 算法的加法同态性，以及实现隐私信息获取的功能进行了验证，并进行了扩展探究，加入了 AES 算法实现简单的对称加密，结果是成功的。实验加深了我对于同态加密的认识，并从代码上直接了解到实际工程中的加密算法是如何实施的；此外对于 AES 等对称加密算法有了一些初步认识。

本次实验的代码已经上传至 GitHub 仓库：[data-security/lab1](https://github.com/data-security/lab1)。